LAPORAN

WORKSHOP SISTEM INFORMASI WEB SERVER (TIF330805)

SEMESTER 3



JSON ENCODING DAN HTTP STATUS CODE

Disusun oleh :

Mochammad Syifaul Qulub (E41250219)

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan aplikasi berbasis web, API (Application Programming Interface) digunakan sebagai penghubung antara aplikasi klien dan server untuk melakukan pertukaran data. Salah satu format yang umum digunakan dalam pertukaran data adalah JSON (JavaScript Object Notation) karena memiliki struktur yang sederhana, ringan, dan mudah diproses oleh berbagai bahasa pemrograman.

Selain format pertukaran data, penggunaan HTTP status code juga diperlukan untuk menunjukkan hasil pemrosesan permintaan yang dikirimkan kepada server. Status code yang tepat membantu klien mengetahui apakah suatu permintaan berhasil atau mengalami kesalahan.

Pada praktikum Acara 18 ini, dilakukan implementasi JSON encoding dan decoding serta penggunaan HTTP status code pada API pengelolaan data mahasiswa menggunakan PHP dan MySQL. Praktikum juga mencakup validasi input, penanganan kesalahan, dan pengujian berbagai endpoint API.

1.2 Tujuan Praktikum

Tujuan pelaksanaan praktikum ini adalah:

1. Memahami konsep JSON encoding dan decoding pada PHP.
2. Mengimplementasikan respons API menggunakan format JSON yang terstruktur.
3. Memahami dan menerapkan HTTP status code sesuai hasil pemrosesan permintaan.
4. Mengimplementasikan validasi input pada API.
5. Menerapkan penanganan kesalahan menggunakan exception handling.
6. Menguji endpoint API untuk operasi GET, POST, PUT, dan DELETE.
7. Memastikan API dapat memberikan respons yang sesuai terhadap permintaan yang berhasil maupun yang mengalami kesalahan.

1.3 Dasar Teori

1.3.1 JSON Encoding dan Decoding

JSON (JavaScript Object Notation) merupakan format pertukaran data yang digunakan untuk mengirimkan informasi antara klien dan server. JSON memiliki format yang mudah dibaca serta dapat diproses oleh berbagai bahasa pemrograman. Dalam PHP, fungsi json_encode() digunakan untuk mengubah data PHP menjadi format JSON, sedangkan json_decode() digunakan untuk membaca dan mengubah data JSON menjadi struktur data PHP.

1.3.2 HTTP Status Code

HTTP status code merupakan kode numerik yang menunjukkan hasil pemrosesan permintaan HTTP oleh server. Beberapa status code yang digunakan dalam praktikum ini adalah:

- **200 OK:** Permintaan berhasil diproses.
- **201 Created:** Data baru berhasil dibuat.
- **400 Bad Request:** Format permintaan atau input tidak valid.
- **404 Not Found:** Endpoint atau data yang diminta tidak ditemukan.
- **405 Method Not Allowed:** HTTP method tidak didukung.
- **409 Conflict:** Terjadi konflik data, seperti NIM duplikat.
- **422 Unprocessable Content:** Input tidak memenuhi aturan validasi.
- **500 Internal Server Error:** Terjadi kesalahan pada server.

1.3.3 Validasi Input

Validasi input merupakan proses pemeriksaan data yang diterima dari pengguna sebelum diproses lebih lanjut. Validasi bertujuan memastikan data memenuhi ketentuan aplikasi, seperti kelengkapan field dan format data yang benar.

Pada praktikum ini, field nim, nama, dan prodi wajib diisi. NIM juga harus terdiri atas tepat sembilan digit angka. Jika beberapa field tidak valid, API dapat mengembalikan seluruh kesalahan tersebut dalam satu respons.

1.3.4 Exception Handling

Exception handling merupakan mekanisme untuk menangani kesalahan yang terjadi selama program berjalan. PHP menyediakan mekanisme try, catch, dan throw untuk membantu mengelola kondisi kesalahan agar aplikasi dapat memberikan respons yang sesuai.

1.3.5 JSON Pretty Print

JSON_PRETTY_PRINT merupakan opsi pada fungsi json_encode() yang membuat keluaran JSON lebih rapi dengan indentasi dan pemisahan baris. Pada praktikum ini, opsi tersebut hanya diaktifkan ketika aplikasi berjalan dalam mode development.

BAB 2. PELAKSANAAN PRAKTIKUM

2.1 Alat dan Bahan

Alat dan bahan yang digunakan dalam pelaksanaan praktikum ini meliputi:

1. Laptop atau komputer.
2. Sistem operasi Windows.
3. Laragon sebagai lingkungan server lokal.
4. Apache sebagai web server.
5. MySQL sebagai sistem manajemen basis data.
6. PHP sebagai bahasa pemrograman.
7. Visual Studio Code sebagai editor kode.
8. Windows PowerShell dan browser untuk pengujian API.

2.2 Persiapan Lingkungan Praktikum

Praktikum dimulai dengan menjalankan Laragon dan mengaktifkan layanan Apache serta MySQL. Selanjutnya, dibuat direktori proyek dengan nama ACARA 18 JSON Encoding dan HTTP Status Code di dalam direktori BKPM WORKSHOP SISTEM INFORMASI WEB SERVER.

Proyek dikembangkan menggunakan PHP native dengan pendekatan Object-Oriented Programming (OOP) dan struktur MVC sederhana untuk memisahkan konfigurasi, model, controller, serta komponen inti aplikasi.

2.3 Pembuatan Basis Data

Basis data dibuat melalui phpMyAdmin dengan nama acara 18. Di dalam basis data tersebut dibuat tabel mahasiswa untuk menyimpan data mahasiswa.

Tabel mahasiswa terdiri atas kolom id sebagai primary key, nim sebagai nomor induk mahasiswa yang bersifat unik, nama untuk menyimpan nama mahasiswa, prodi untuk menyimpan program studi, serta created_at untuk mencatat waktu pembuatan data.

Beberapa data awal dimasukkan ke dalam tabel untuk mendukung pengujian operasi pengambilan, penambahan, perubahan, dan penghapusan data.

2.4 Implementasi Konfigurasi Basis Data

File app/config/Database.php digunakan untuk mengatur koneksi antara aplikasi PHP dan MySQL menggunakan PDO. Konfigurasi meliputi host, nama basis data, username, password, dan charset.

PDO dikonfigurasi agar kesalahan basis data menghasilkan exception, hasil query dikembalikan sebagai associative array, dan prepared statement native digunakan. Dengan konfigurasi ini, proses pengelolaan data dapat dilakukan secara terstruktur.

2.5 Implementasi Autoloader

File app/core/autoload.php digunakan untuk memuat kelas PHP secara otomatis ketika dibutuhkan. Autoloader memeriksa direktori konfigurasi, core, model, dan controller.

Penggunaan autoloader membantu mengurangi pemanggilan file secara manual dan membuat pengelolaan kelas dalam proyek menjadi lebih terstruktur.

2.6 Implementasi Kelas Response

File app/core/Response.php digunakan untuk menyeragamkan format respons API. Respons yang dihasilkan mencakup lima bagian, yaitu success, message, data, errors, dan timestamp.

Fungsi http_response_code() digunakan untuk menentukan HTTP status code, sedangkan json_encode() digunakan untuk mengubah data menjadi JSON. Opsi JSON_UNESCAPED_SLASHES dan JSON_UNESCAPED_UNICODE digunakan untuk menghasilkan keluaran yang lebih mudah dibaca.

Selain itu, JSON_PRETTY_PRINT hanya ditambahkan ketika aplikasi berjalan dalam mode development.

2.7 Implementasi Model Mahasiswa

File app/models/Mahasiswa.php berfungsi sebagai penghubung antara aplikasi dan tabel mahasiswa. Model menyediakan fungsi untuk mengambil seluruh

data, mencari data berdasarkan ID, menambahkan data baru, memperbarui data, dan menghapus data.

Operasi basis data dilakukan menggunakan PDO dan prepared statement. Pendekatan tersebut membantu menjaga keamanan pemrosesan input serta memisahkan logika akses data dari controller.

2.8 Implementasi Controller Mahasiswa

File app/controllers/MahasiswaController.php bertanggung jawab memproses permintaan API yang berkaitan dengan data mahasiswa.

Controller menyediakan fungsi index(), store(), update(), dan destroy() untuk menangani operasi GET, POST, PUT, dan DELETE. Selain itu, controller melakukan validasi input JSON, memeriksa kelengkapan field, memvalidasi format NIM, serta menangani kondisi data tidak ditemukan dan konflik NIM.

Jika beberapa field wajib tidak valid, controller mengumpulkan kesalahan tersebut dalam array errors sehingga beberapa pesan kesalahan dapat dikembalikan sekaligus dalam satu respons.

2.9 Implementasi Router

File app/core/Router.php digunakan untuk mengarahkan permintaan berdasarkan path URL dan HTTP method yang digunakan.

Endpoint /mahasiswa diarahkan ke MahasiswaController. Permintaan GET diproses melalui index(), POST melalui store(), PUT melalui update(), dan DELETE melalui destroy().

Jika endpoint tidak ditemukan, router mengembalikan status 404. Jika HTTP method tidak didukung, router mengembalikan status 405 beserta informasi metode yang diperbolehkan.

2.10 Implementasi Entry Point dan URL Rewriting

File public/index.php menjadi titik masuk aplikasi. File ini memuat autoloader, menjalankan router, dan menangani kesalahan yang tidak tertangani agar server dapat mengembalikan respons JSON dengan status 500.

File public/.htaccess digunakan untuk mengatur URL rewriting sehingga permintaan yang bukan file atau direktori fisik diteruskan ke index.php.

2.11 Pengujian Endpoint GET

Pengujian GET dilakukan untuk memastikan API dapat mengambil seluruh data mahasiswa dan mencari data berdasarkan ID tertentu.

Endpoint yang diuji adalah:

GET /public/mahasiswa

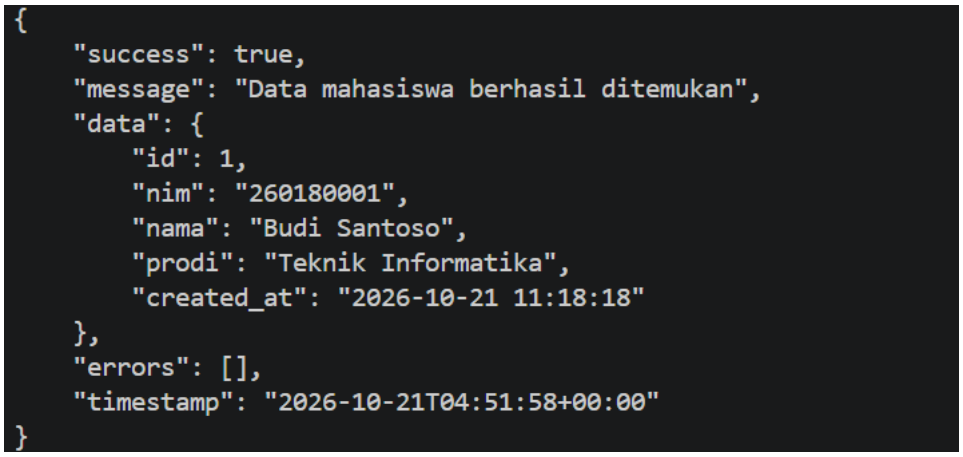
GET /public/mahasiswa?id=6

Hasil pengujian menunjukkan bahwa API dapat mengembalikan data mahasiswa dalam format JSON dengan status 200 OK.

Gambar 2.5 Pengujian GET seluruh data mahasiswa


```
{
  "success": true,
  "message": "Data mahasiswa berhasil diambil",
  "data": [
    {
      "id": 3,
      "nim": "260180003",
      "nama": "Andi Pratama",
      "prodi": "Manajemen Informatika",
      "created_at": "2026-10-21 11:18:18"
    },
    {
      "id": 2,
      "nim": "260180002",
      "nama": "Siti Aminah",
      "prodi": "Teknik Informatika",
      "created_at": "2026-10-21 11:18:18"
    },
    {
      "id": 1,
      "nim": "260180001",
      "nama": "Budi Santoso",
      "prodi": "Teknik Informatika",
      "created_at": "2026-10-21 11:18:18"
    }
  ],
  "errors": [],
  "timestamp": "2026-10-21T04:22:24+00:00"
}
```

Gambar 2.6 Pengujian GET berdasarkan ID

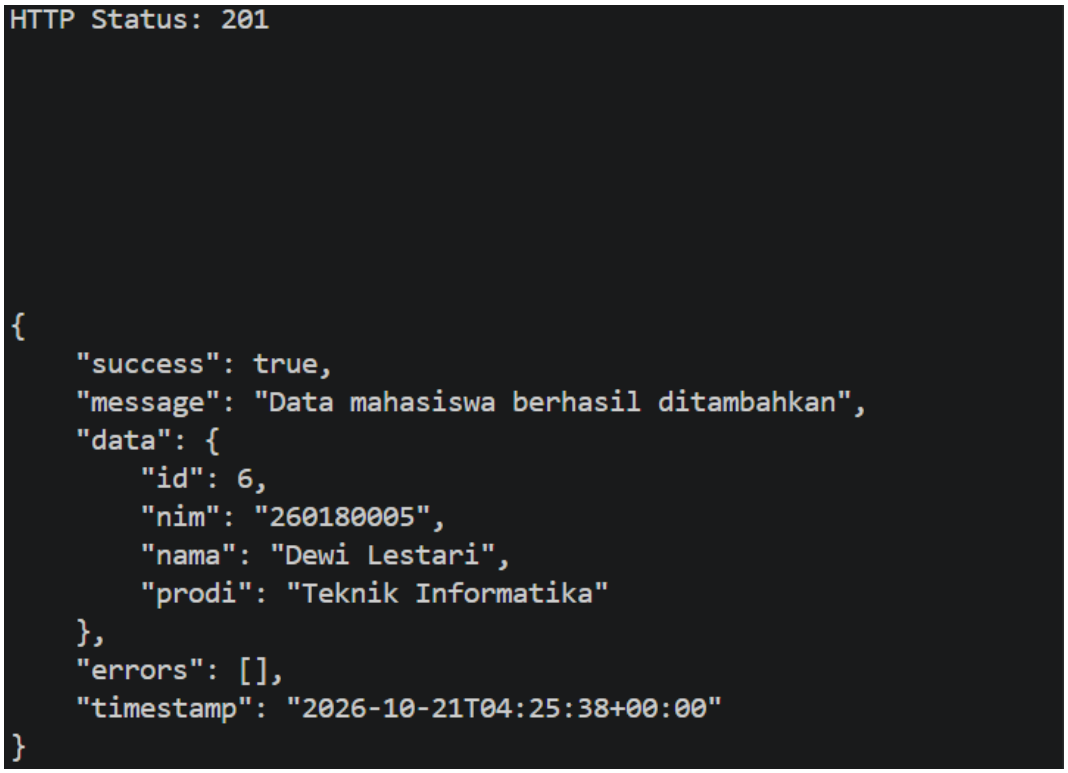


2.12 Pengujian Endpoint POST

Pengujian POST dilakukan untuk menambahkan data mahasiswa baru melalui body permintaan dalam format JSON. Data yang dikirimkan mencakup NIM, nama, dan program studi.

Ketika penambahan data berhasil, API mengembalikan status 201 Created beserta informasi data yang ditambahkan. Pengujian juga mencakup penggunaan NIM yang sudah terdaftar untuk memastikan API dapat menangani konflik data dengan status 409 Conflict.

Gambar 2.7 Pengujian POST penambahan data mahasiswa



```
HTTP Status: 201

{
    "success": true,
    "message": "Data mahasiswa berhasil ditambahkan",
    "data": {
        "id": 6,
        "nim": "260180005",
        "nama": "Dewi Lestari",
        "prodi": "Teknik Informatika"
    },
    "errors": [],
    "timestamp": "2026-10-21T04:25:38+00:00"
}
```

Gambar 2.8 Pengujian POST dengan NIM duplikat



```
HTTP Status: 409

{
    "success": false,
    "message": "NIM sudah digunakan",
    "data": null,
    "errors": {
        "nim": "Gunakan NIM yang berbeda"
    },
    "timestamp": "2026-10-21T04:53:51+00:00"
}
```

2.13 Pengujian Validasi Input

Validasi input dilakukan dengan mengirimkan data yang tidak sesuai dengan ketentuan. Pengujian mencakup NIM yang tidak terdiri atas sembilan digit angka, field wajib yang kosong, serta format JSON yang tidak valid.

API memberikan respons sesuai jenis kesalahan. Input dengan format JSON yang tidak valid menghasilkan status 400 Bad Request, sedangkan NIM yang tidak memenuhi aturan format menghasilkan status 422 Unprocessable Content.

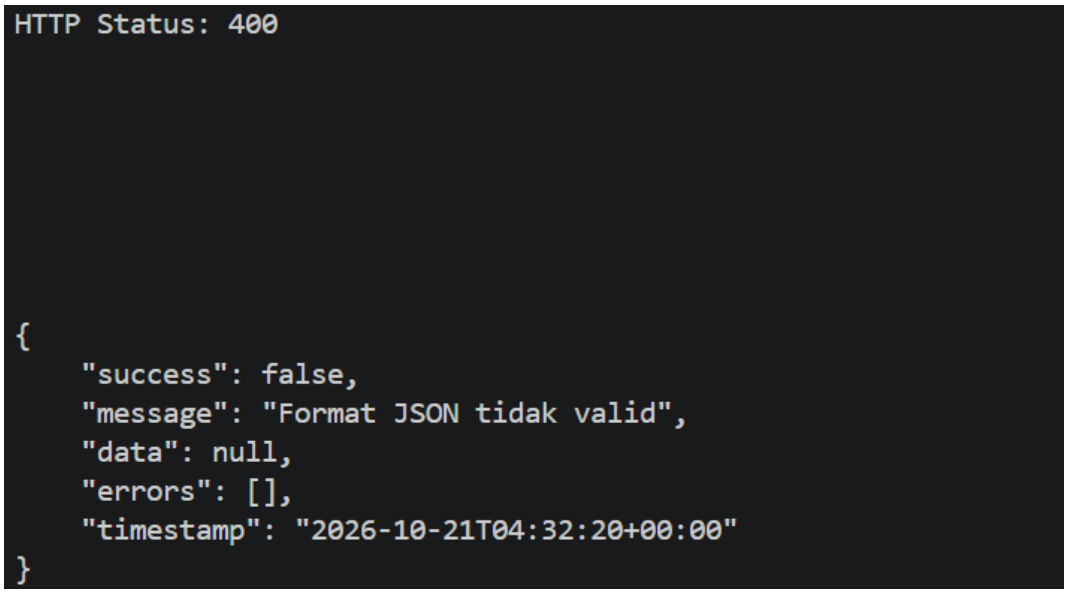
Gambar 2.9 Pengujian validasi format NIM



Gambar 2.10 Pengujian beberapa kesalahan field sekaligus



Gambar 2.11 Pengujian format JSON tidak valid



2.14 Pengujian Endpoint PUT

Pengujian PUT dilakukan untuk memperbarui informasi mahasiswa berdasarkan ID. Data yang dikirimkan diproses melalui controller dan model, kemudian API mengembalikan data terbaru setelah pembaruan berhasil.

Hasil pengujian menunjukkan bahwa data mahasiswa dapat diperbarui dan API memberikan status 200 OK.

Gambar 2.12 Pengujian PUT pembaruan data mahasiswa

```
PS C:\laragon\www\BKPM WORKSHOP SISTEM INFORMASI WEB SERVER\ACARA
 18 JSON Encoding dan HTTP Status Code>
>> Invoke-WebRequest -Uri "${uri}?id=6" -Method GET -UseBasicPars
ing | Select-Object StatusCode, Content
>>

StatusCode Content
---------- -------
       200 ...
```

2.15 Pengujian Endpoint DELETE

Pengujian DELETE dilakukan untuk menghapus data mahasiswa berdasarkan ID. Sebelum penghapusan dilakukan, API memeriksa keberadaan data tersebut. Jika data ditemukan dan berhasil dihapus, API mengembalikan pesan keberhasilan beserta ID data yang dihapus.

Gambar 2.13 Pengujian DELETE data mahasiswa

```
HTTP Status: 200

{
    "success": true,
    "message": "Data mahasiswa berhasil dihapus",
    "data": {
        "id": 6
    },
    "errors": [],
    "timestamp": "2026-10-21T04:30:10+00:00"
}
```

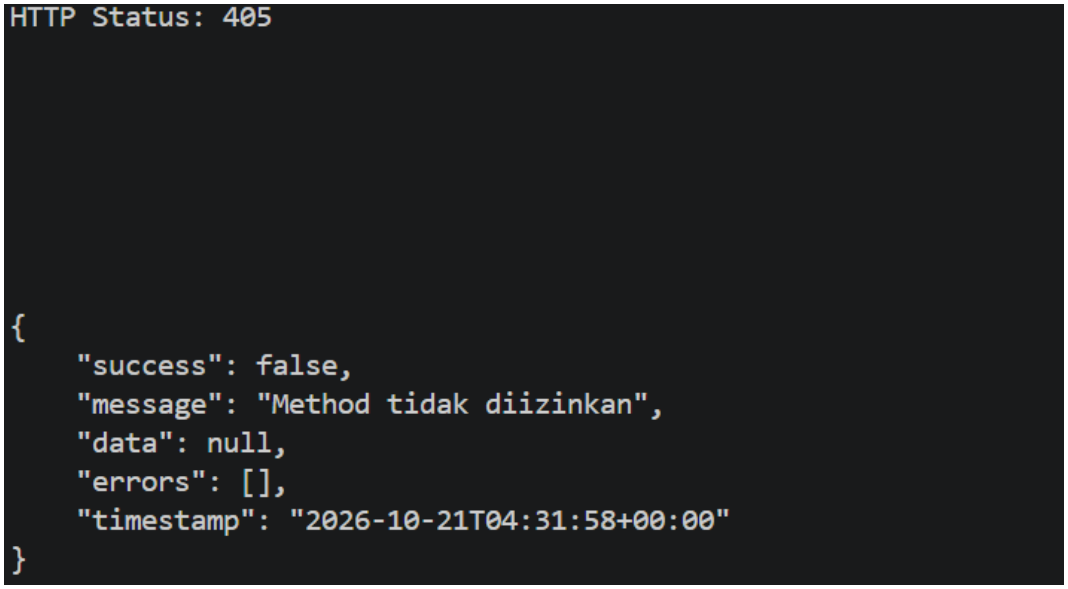
2.16 Pengujian HTTP Status Code

Pengujian HTTP status code dilakukan untuk memastikan API memberikan kode yang sesuai terhadap berbagai kondisi permintaan.

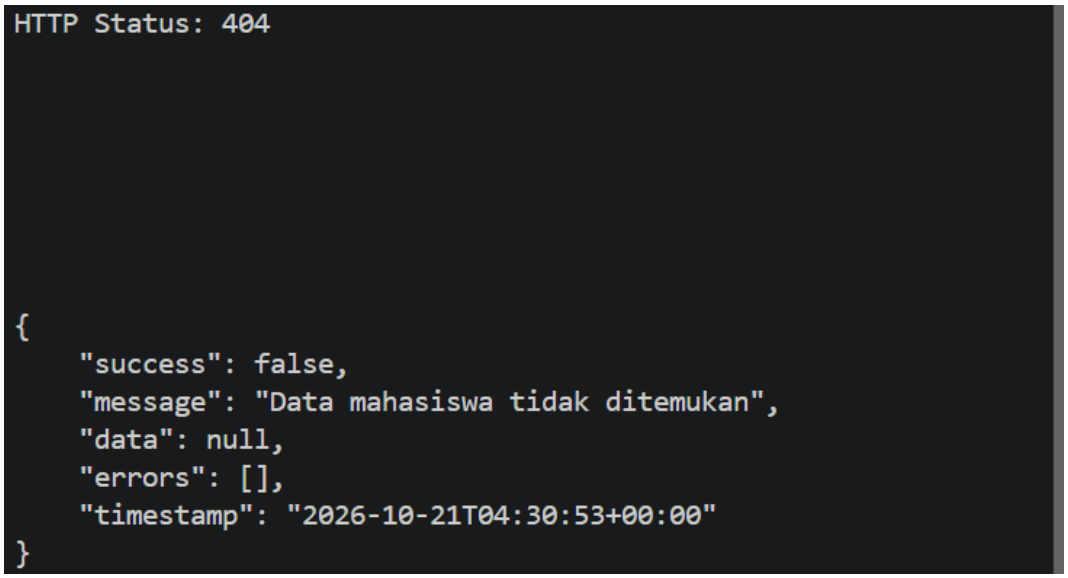
Hasil pengujian yang telah dilakukan meliputi:

Kondisi Pengujian	HTTP Status Code
Pengambilan data berhasil	200 OK
Penambahan data berhasil	201 Created
Format JSON tidak valid	400 Bad Request
Data tidak ditemukan	404 Not Found
HTTP method tidak didukung	405 Method Not Allowed
NIM duplikat	409 Conflict
Format NIM tidak sesuai	422 Unprocessable Content

Gambar 2.14 Pengujian HTTP method yang tidak didukung



Gambar 2.15 Pengujian data mahasiswa tidak ditemukan



2.17 Hasil Praktikum

Berdasarkan implementasi dan pengujian yang telah dilakukan, API pengelolaan data mahasiswa dapat menjalankan operasi CRUD menggunakan HTTP method GET, POST, PUT, dan DELETE. API menghasilkan respons JSON dengan struktur yang konsisten dan menggunakan HTTP status code sesuai kondisi pemrosesan.

Validasi input dapat menangani field wajib yang kosong, format NIM yang tidak sesuai, serta JSON yang tidak valid. API juga dapat menangani konflik NIM, data yang tidak ditemukan, dan HTTP method yang tidak didukung.

Penggunaan JSON_PRETTY_PRINT telah diatur agar hanya aktif pada mode development. Dengan demikian, implementasi telah mencakup kebutuhan utama praktikum Acara 18 mengenai JSON encoding dan HTTP status code.

BAB 3. PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum Acara 18 yang telah dilaksanakan, dapat disimpulkan bahwa JSON encoding dan HTTP status code merupakan komponen penting dalam pengembangan API. JSON digunakan sebagai format pertukaran data, sedangkan HTTP status code memberikan informasi mengenai hasil pemrosesan permintaan oleh server.

Implementasi API menggunakan PHP, PDO, dan struktur MVC sederhana berhasil mendukung operasi CRUD terhadap data mahasiswa. Selain itu, validasi input dan penanganan kesalahan diterapkan untuk menjaga konsistensi respons serta memberikan informasi yang sesuai ketika terjadi kesalahan.

Melalui praktikum ini, diperoleh pemahaman mengenai pembuatan respons JSON yang terstruktur, penggunaan HTTP status code yang tepat, validasi data, dan penanganan kesalahan dalam pengembangan API berbasis web.

3.2 Saran

Dalam pengembangan API selanjutnya, validasi input perlu terus ditingkatkan untuk menangani berbagai kemungkinan data yang tidak sesuai. Pengujian terhadap seluruh endpoint dan HTTP status code juga perlu dilakukan secara konsisten agar aplikasi dapat memberikan respons yang tepat.

Selain itu, konfigurasi lingkungan development dan production perlu dipisahkan dengan baik untuk mendukung keamanan, kemudahan debugging, dan efisiensi respons API.